WIRESHARK COMPUTER NETWORKS LAB

PRACTICAL STUDY GUIDE

Questions discussed in this chat, where to click in Wireshark, filters, packet fields, and calculations.

HTTP | DNS | UDP | TCP

How to use this guide

This is a practical, step-by-step companion for the lab questions covered in our chat. It is written so you can repeat the method in an exam or lab viva instead of memorising one fixed answer.

Golden rule: First identify the correct packet. Then expand the correct protocol in the middle Packet Details pane. Read the named field from that packet. Do not copy a value from a different trace or from your current network connection.

Contents

Section	What it covers
1. Wireshark basics	Packet panes, filters, time, printing
2. Week 0 HTTP familiarisation	Protocols, GET-to-OK delay, IP addresses, printing
3. DNS exercises	Queries, responses, ports, A/PTR/NS, cache
4. UDP exercise	Header fields, length, max values, ports
5. HTTP exercises	Headers, status codes, conditional GET, objects, authentication
6. TCP assignment	Handshake, POST, sequence numbers, ACKs, RTT, windows, retransmission, throughput
7. Fast exam sheet	Formulas, common filters, checklist

Trace-specific values included

Examples explicitly marked as coming from the traces/data shared in this chat include the gaia server address 128.119.245.12, client address 192.168.1.102, TCP port 1161, the first six data-segment sequence numbers and lengths, DNS example answers, and the ACK-number differences you provided. Other answers must be read from the actual packet file.

1. Wireshark basics: know where to look

Wireshark is a packet analyzer used to capture, inspect and analyse network traffic. It normally has three main panes: Packet List (top), Packet Details (middle), and Packet Bytes (bottom).

Pane	What you use it for
Packet List	Find the frame/packet, see Time, Source, Destination, Protocol, Length and Info.
Packet Details	Expand Ethernet, IP, TCP/UDP, HTTP or DNS and read exact header fields.
Packet Bytes	Inspect the actual bytes/payload. Useful for finding text such as POST when protocol decoding is not obvious.

Basic workflow

1. Open the correct .pcap or .pcapng file with File > Open.

2. Click a packet in the top Packet List.

3. In the middle pane, click the small arrow beside the protocol name, such as Transmission Control Protocol or Domain Name System.

4. Read the exact field named in the question. For TCP data length, look for [TCP Segment Len], not the full Ethernet frame length.

Display filters you can reuse

Purpose	Display filter
All HTTP packets	http
DNS packets	dns
UDP packets	udp
TCP packets	tcp
Only one TCP stream	tcp.stream eq 0
SYN packets	tcp.flags.syn == 1
SYN and ACK packets	tcp.flags.syn == 1 && tcp.flags.ack == 1
ACK packets	tcp.flags.ack == 1
HTTP POST requests	http.request.method == "POST"
TCP retransmissions	tcp.analysis.retransmission || tcp.analysis.fast_retransmission
Zero-window advertisements	tcp.window_size_value == 0
Packets from a specific IP	ip.src == 128.119.245.12
Packets to/from port 80	tcp.port == 80

Important: A display filter hides packets that do not match; it does not delete them. Clear the filter to return to the full trace. Filters are shortcuts, not something you must memorise for a written exam.

Finding a text string such as POST without memorising a filter

1. Press Ctrl+F.

2. Choose Find By: String and Search In: Packet bytes.

3. Enter POST and click Find.

4. Select the matching packet. Expand Transmission Control Protocol to read Sequence Number and [TCP Segment Len].

Time display

The Time column is commonly shown as seconds since the capture started. For elapsed-time calculations, this is convenient. To show clock time, use View > Time Display Format > Time-of-day. Always subtract times from the same capture and same time format.

2. Week 0: Wireshark / HTTP familiarisation

Q1. List three protocols in the unfiltered packet list

Clear all display filters. Look down the Protocol column in the Packet List and write any three distinct protocols that actually appear in your trace (for example DNS, TCP, HTTP, UDP, TLS, ARP, depending on the capture). Do not guess protocols that are not shown.

Q2(b). How long from HTTP GET until HTTP OK?

1. Find the HTTP GET request in the Packet List. Select it and note its Time value.

2. Find the matching HTTP response, normally with status 200 OK. Make sure it belongs to the same request/connection.

3. Note the response Time.

Calculation: Delay = response Time - GET Time. The result is in seconds if the Time column is in seconds since capture start.

Notebook answer format: The time between the HTTP GET and the corresponding HTTP 200 OK response was ___ seconds.

Q2(c). IP address of gaia.cs.umass.edu and your computer

Select the HTTP GET to gaia.cs.umass.edu. In the Internet Protocol Version 4 section, read Source and Destination. When the client sends the GET, the destination is the server (gaia) and the source is the client. In the trace discussed in this chat, gaia was **128.119.245.12** and the client was **192.168.1.102**. Your live ipconfig address can be different from the address in a supplied historical trace.

Q2(d). Print the GET and OK messages

Select the GET packet, then File > Print. Choose Selected Packet Only and Print as displayed. Repeat for the matching OK response. If you do not have a printer, print to PDF if that option is available.

Exam note: matching matters

Do not subtract the time of an unrelated GET and response. Match the request and reply by the same TCP stream, HTTP request/response, server, and requested object. A Microsoft connect-test request is not the same as a gaia lab request.

3. DNS exercises: query, response, records and cache

A DNS exchange usually has a query and a response. DNS commonly uses UDP port 53. In a query, destination port is 53; in the reply, source port is 53. The IP addresses reverse direction in the response.

Q. What transport protocol is used? What are the ports?

Select a DNS query. Expand User Datagram Protocol. Read Source Port and Destination Port. For a normal DNS query, Destination Port = 53. Select the response: Source Port = 53 and Destination Port is the client-side temporary port. Check the actual trace; DNS can also use TCP in some situations.

Q. What DNS server did the host query? Compare with ipconfig /all

Select the DNS query and look at the IP header. The query destination IP is the DNS server address used by that packet. On Windows, open Command Prompt and run `ipconfig /all`; find DNS Servers under the active network adapter. Compare only when the ipconfig data belongs to the same computer/network context as the capture. A live Wi-Fi/hotspot DNS address should not be compared as though it were the server in an old supplied trace.

Q. What type of query is it, and how many Answer RRs?

1. Select the DNS query. Expand Domain Name System (query) > Queries.

2. Read the queried name and Type (A, AAAA, PTR, NS, etc.). Query packets usually show Answer RRs = 0.

3. Select the matching DNS response. Read Answer RRs and expand Answers to see the returned records.

Common DNS record types

Type	Meaning / what to recognise
A	Maps a host name to an IPv4 address.
AAAA	Maps a host name to an IPv6 address.
PTR	Reverse lookup: maps an IP-style reverse name to a host name.
NS	Lists authoritative name servers for a domain.
CNAME	Alias name that points to another canonical name.

Q. Reverse lookup / PTR response

A PTR query name often ends in `in-addr.arpa` for IPv4 reverse lookups. In the response, expand Answers and read the PTR name. In the trace discussed, a reverse lookup response returned **dns-prime.poly.edu** as a PTR name.

Q. What does “No such name” mean?

Select the response and check the DNS flags / response code. “No such name” is NXDOMAIN: the queried name does not exist. In the example discussed, `www.mit.edu.poly.edu` received a no-such-name response, followed by a successful query for `www.mit.edu`.

Q. Which MIT name servers are listed? Are their IP addresses included?

Select the response to the NS query for mit.edu. Expand Answer RRs / Answers and read the NS names. Then expand Additional RRs / Additional records and check for A records that provide their IPv4 addresses. In the trace discussed, the NS names were **bitsy.mit.edu**, **strawb.mit.edu**, and **w20ns.mit.edu**; additional A records were **18.72.0.3**, **18.71.0.151**, and **18.70.0.160**.

Q. Does the TCP SYN destination IP match an address returned by DNS?

Find the DNS response and expand Answers to note the A record IP address. Then find the initial TCP SYN to that host (SYN = 1 and ACK = 0) and read its IP Destination. Compare the SYN destination IP with the DNS A record address. In the trace discussed, the SYN destination matched one of the DNS response IP addresses.

Q. Does an image on a page always cause a new DNS query?

No. The browser may already have the hostname-to-IP mapping in its DNS cache, so another DNS query may not appear. The browser can still send a separate HTTP GET for the image. DNS caching explains why a page image can load without a new DNS request.

Examples from your traces: DNS trace 1, Frame 8 queried www.ietf.org (A; Answer RRs 0) and Frame 9 returned 2 answers via DNS endpoint 128.238.29.23. DNS trace 2, Frames 15-16 performed a PTR lookup and returned dns-prime.poly.edu via 128.238.29.22; Frames 17-18 returned No such name for www.mit.edu.poly.edu; Frames 19-20 returned A = 18.7.22.83 for www.mit.edu.

4. UDP exercise: header fields and calculations

Q1. Name the four UDP header fields

Source Port, Destination Port, Length, and Checksum. These are the four fields in the UDP header.

Q2. How many bytes are in each UDP header field?

Each field is 2 bytes (16 bits). Four fields x 2 bytes = an 8-byte UDP header.

Q3. What does the UDP Length field measure?

It is the length of the entire UDP datagram: UDP header plus UDP payload. For the sample frame in the chat, UDP Length = 51 bytes and payload = 43 bytes, so 8 + 43 = 51 bytes.

Q4. Maximum UDP payload length

The UDP Length field is 16 bits, so its theoretical maximum is 65,535 bytes. Subtract the 8-byte UDP header: maximum UDP payload = **65,527 bytes**. For IPv4, the practical maximum is lower because the IP header also occupies space (commonly 65,507 bytes with a minimum 20-byte IPv4 header). Use 65,527 when the question specifically asks from the UDP Length field.

Q5. Maximum source port number

A UDP port field is 16 bits, so the maximum value is **65,535**.

Q6. IP protocol number for UDP

UDP is IP protocol number **17** in decimal, or **0x11** in hexadecimal. Find it in the IPv4 Protocol field.

Q7. Compare request and response ports

For a request/reply exchange, source and destination ports normally swap. Example from the chat: request source port 4372, destination port 53; reply source port 53, destination port 4372. Also check the source and destination IP addresses reverse.

Where to find UDP fields

Select the frame, expand User Datagram Protocol in Packet Details, then read Source Port, Destination Port, Length, and Checksum. Do not use the Ethernet frame length as UDP Length.

5. HTTP exercises: headers, responses and objects

For HTTP questions, first filter with `http` or find the relevant GET request. Select the packet and expand Hypertext Transfer Protocol in Packet Details. The request contains browser headers; the response contains status line and response headers.

Q. What HTTP version does the browser use? What version does the server use?

Read the request line (for example, GET /... HTTP/1.1) in the request packet. Read the status line (for example, HTTP/1.1 200 OK) in the response packet.

Q. What language does the browser prefer?

In the GET request, find the Accept-Language header. Write its exact value. In the trace discussed, it was **en-us**, indicating English (US).

Q. What are the client and server IP addresses?

Select the GET request and read the IPv4 Source and Destination. Client = source; server = destination for an outbound request. In the HTTP trace discussed, client = **192.168.1.102** and gaia server = **128.119.245.12**.

Q. What status code did the server return?

Read the first line of the HTTP response. Examples: **200 OK** means success; **304 Not Modified** means the cached copy can be used; **401 Unauthorized** means authentication is required.

Q. Last-Modified and Content-Length

In the HTTP response packet, expand the response headers. Find `Last-Modified:` and copy the exact date/time. Find `Content-Length:` and copy the exact byte count. These values are specific to the response; they were not provided in the trace text shared in chat, so do not make them up.

Q. Why do packet list summaries omit information?

The packet list is a summary. Select the HTTP packet and expand the HTTP details, or inspect Packet Bytes, to see full request/response headers such as Date, Server, Content-Type, Last-Modified, and Content-Length when present.

Conditional GET questions

1. Find the first GET request and check whether it contains If-Modified-Since. If the header is absent, answer No.

2. Check the first response. A 200 OK response with the object body means the server sent the object.

3. Find the second GET. Check its If-Modified-Since header and copy the exact date/time value.

4. Read the following response status. A 304 Not Modified tells the browser its cached copy is still valid; the server does not send the object body again.

HTTP trace discussed: first response was 200 OK; second response was 304 Not Modified. The exact If-Modified-Since date must be copied from the second GET packet in your trace.

Long document: how many TCP data segments carry the response?

Select the HTTP response, then inspect the TCP packets in the same stream that carry data. Count data-carrying segments, and read each [TCP Segment Len]. In the long-document trace discussed, four data-containing TCP segments carried 1,460 + 1,460 + 1,460 + 436 = **4,816 bytes**.

Embedded objects: how many GETs, and are images downloaded in parallel?

Filter `http.request.method == "GET"`. Count the GET requests for the HTML page and embedded files. For each GET, inspect destination IP and TCP stream. To decide whether downloads overlap, compare timestamps: if multiple GETs on different TCP connections are sent before the first object finishes arriving, the browser is downloading objects in parallel. In the trace discussed, three GETs fetched one HTML page and two images: Frame 10 for /ethereal-labs/lab2-4.html to 128.119.245.12, Frame 17 for /catalog/images/pearson-logo-footer.gif to 165.193.123.218, and Frame 20 for /~kurose/cover.jpg to 134.241.6.82. The two image requests were close together on separate TCP connections: Frame 17 at 11:08:41.756098 and Frame 20 at 11:08:41.759416. Frame 25 completed the first image response at 11:08:41.783667, while the second response continued until Frame 54 at 11:08:42.040490; the overlap demonstrates parallel downloading.

6. TCP assignment: handshake, sequence numbers, ACKs and performance

Trace used in the chat: client 192.168.1.102, gaia.cs.umass.edu 128.119.245.12, client TCP port 1161, server TCP port 80. These are the values from tcp-ethereal-trace-1, not necessarily the IP/port values of another capture.

Q1-Q2. Identify client and server addresses and TCP ports

Select the first client SYN packet. Read IPv4 Source/Destination and TCP Source Port/Destination Port. Client SYN: source IP and source port are the client; destination IP and destination port are the server. In the trace shared, client = 192.168.1.102:1161 and gaia = 128.119.245.12:80.

Q3. Sequence number of SYN and what identifies it as SYN?

Find the first TCP packet with SYN set, normally the first packet in the connection. Expand Transmission Control Protocol. Read Sequence Number. Expand Flags: SYN must be Set (SYN = 1). In the shared trace, Wireshark relative sequence number for the client SYN was **0**.

Q4. SYNACK sequence and acknowledgement number; why?

Select the server reply. It must have both SYN = 1 and ACK = 1 in TCP Flags. Read its Sequence Number and Acknowledgment Number. The ACK number is the client initial sequence number + 1 because a SYN consumes one sequence number. In the shared trace using relative numbering: server SYNACK Seq = **0**, Ack = **1**. Raw sequence numbers are different; use the numbering mode asked for / shown by Wireshark.

Q5. Find the TCP segment containing HTTP POST

1. Use Ctrl+F > String > Packet bytes > POST, or filter with http.request.method == "POST" if HTTP is decoded.

2. Select the matching packet and expand Transmission Control Protocol.

3. Read Sequence Number. Do not confuse Frame number with TCP Sequence Number.

In the trace shared, the POST-containing first data segment was Frame 4 and its relative TCP sequence number was **1**.

Q6. First six data segments: sequence numbers, send time, ACK time, RTT

Count the POST-containing data segment as segment 1. Do not count SYN or SYNACK. For each of the six outbound data packets, note Frame number, Time, Sequence Number, and [TCP Segment Len]. The first six data frames identified from the shared trace were 4, 5, 7, 8, 10, and 11.

Segment	Frame	Relative Seq	TCP data length
1 (POST)	4	1	565 bytes
2	5	566	1460 bytes
3	7	2026	1460 bytes
4	8	3486	1460 bytes
5	10	4946	1460 bytes
6	11	6406	1460 bytes

Fill in this table from your own packet timestamps

Segment	Send Time	ACK Time	RTT
1 (Frame 4)			
2 (Frame 5)			
3 (Frame 7)			
4 (Frame 8)			
5 (Frame 10)			
6 (Frame 11)			

How to find each send time

Read the Time column for the six client-to-server data packets. If the Time column is hidden, right-click a column header and use Column Preferences, or widen the column. Use the same capture time format for all six.

How to match the ACK time

1. For a data segment, calculate the ending byte sequence number: Sequence Number + TCP Segment Len.

2. Find a packet travelling in the reverse direction (server to client).

3. Read its Acknowledgment Number. The first ACK whose number is equal to or greater than the segment ending sequence number acknowledges that segment.

4. Read that ACK packet Time. That is the ACK receive time for the segment (a cumulative ACK may acknowledge more than one segment).

RTT formula: RTT = ACK receive time - data segment send time. Example only: if sent at 0.500 s and acknowledged at 0.525 s, RTT = 0.025 s = 25 ms. Do not use the example times as trace answers.

Shortcut graph

Select a packet in the TCP stream, then Statistics > TCP Stream Graph > Round Trip Time Graph. The graph plots RTT samples for segments in that stream. Use it to check your manual calculations; it does not remove the need to identify the correct six segments.

Q7. Length of each of the first six TCP segments

Read [TCP Segment Len] for each of the six outbound data packets. Do not use Frame length (Ethernet + IP + TCP headers + data). In the shared trace, the six lengths were **565, 1460, 1460, 1460, 1460, and 1460 bytes**.

Q8. Minimum advertised receive buffer/window; does it throttle the sender?

1. Identify packets from the receiver/server back to the sender/client. In this trace, server source IP is 128.119.245.12.

2. Expand TCP and read Window or [Calculated window size]. Prefer calculated window size when window scaling is in use.

3. Scan all server-to-client packets in this connection and take the smallest advertised value.

4. Check whether the receiver ever advertises a zero window. A zero window is strong evidence that receiver buffer space is stopping the sender; look for TCP ZeroWindow / persist-probe behaviour. A merely small non-zero window is not by itself proof of throttling.

Caution: Packet 199 in the chat was client-to-server and showed Window = 17,520. That is the client advertising its receive window, not the server value required if the question asks for the receiver/server. Packet 2 (server SYNACK) showed Window = 5,840, but you must scan the whole server direction to find the trace minimum.

Q9. Are any segments retransmitted? What did you check?

Use one of these display filters: `tcp.analysis.retransmission` or `tcp.analysis.fast_retransmission`. Combined: `tcp.analysis.retransmission || tcp.analysis.fast_retransmission`. You can also inspect the Info column for labels such as TCP Retransmission / TCP Fast Retransmission. If no matching packets appear, answer that none were identified by these indicators. Only state Yes/No after checking the actual trace.

Q10. How many bytes does the receiver typically acknowledge? Does it ACK every other segment?

Look at consecutive server-to-client ACK packets. Read Acknowledgment Number from each. Subtract the previous ACK number from the current ACK number. The difference is the newly acknowledged data bytes. Ignore the SYNACK handshake when analysing data ACK behaviour.

ACK number	Difference from previous ACK	Newly acknowledged bytes
566	566 - 1	565
2026	2026 - 566	1460
3486	3486 - 2026	1460
4946	4946 - 3486	1460
7866	7866 - 4946	2920
9013	9013 - 7866	1147
10473	10473 - 9013	1460

Based on the ACK numbers you supplied in chat, most increments are 1460 bytes. One increment is 2920 bytes (7866 - 4946), which equals two 1460-byte segments. That is a case consistent with ACKing every other full-sized segment. Other increments can differ when the final segment is shorter or ACKs are cumulative.

Q11. TCP throughput (bytes per unit time)

Throughput = total TCP payload bytes transferred / elapsed transfer time. Sum the TCP data lengths (TCP Segment Len / tcp.len) for the data direction being measured. Use the first data segment time and last data segment time for the transfer interval, unless your lab instructions specify ending at the final ACK. Keep units consistent.

Example method for the shared trace

1. Find the first data segment (POST, Frame 4) and note its Time.

2. Find the final data-carrying segment in the upload and note its Time.

3. Duration = final data-segment Time - first data-segment Time.

4. Divide the total payload bytes by this duration. Result is bytes/second. Multiply by 8 for bits/second.

The Packet 199 detail shared in chat displayed a reassembled TCP data total of 164,090 bytes. Use this only if that is the total application data for the transfer being measured. You still need the actual start/end timestamps from your trace to calculate a defensible throughput value.

7. Fast exam sheet: formulas and habits

Formulas

What you need	Formula / rule
GET-to-OK delay	Response Time - GET Time
Next TCP sequence number	Current Sequence Number + TCP Segment Len
ACK bytes	Current ACK Number - Previous ACK Number
RTT	ACK receive Time - segment send Time
UDP payload length	UDP Length - 8 bytes
Theoretical UDP max payload	65,535 - 8 = 65,527 bytes
TCP throughput	Total TCP payload bytes / elapsed seconds
Bits per second	Bytes per second x 8

TCP flags quick recognition

Packet type	Flags to look for
SYN	SYN = 1, ACK usually 0 for initial SYN
SYNACK	SYN = 1 and ACK = 1
ACK	ACK = 1; SYN and FIN may be 0
FIN	FIN = 1; used to close a connection
RST	RST = 1; resets a connection

Common mistakes to avoid

- Do not confuse Frame number, Stream Packet Number, Sequence Number, and Acknowledgment Number.
- Do not confuse full frame length with TCP Segment Len or UDP Length.
- Do not use the client receive Window when the question asks for the server/receiver window.
- Do not count the SYN/SYNACK handshake as one of the first six data segments when the question says the POST segment is first.
- Do not assume every ACK acknowledges two segments. Calculate the difference between consecutive ACK numbers.
- Do not state an exact timestamp, RTT, minimum window, retransmission result, or throughput without checking the trace.
- Relative TCP sequence numbers are easier to read; raw sequence numbers are different. Be clear which one you report.
- HTTP authentication: look for a 401 Unauthorized response and a later request containing an Authorization header; verify in the trace.

Useful Windows shortcuts

Shortcut	Use
Ctrl + F	Find a text string such as POST in packet bytes.
Ctrl + G	Go to a frame number.
Ctrl + Shift + S	Save As in many applications; not needed for packet analysis.
Windows + Shift + S	Take a screenshot of the Wireshark area.

Last-minute practical routine

1. Open the correct trace file and clear old filters.

2. Identify the direction using Source and Destination IPs.

3. Expand the right protocol and read the exact named field.

4. Write down packet number, Time, Sequence/Acknowledgment Number, and data length when relevant.

5. Do arithmetic only after recording the fields. Include units in the final answer.

Final reminder: Follow the exact wording of the question and verify trace-specific values in the assigned capture file.